

MAKALAH
***REFACTORING* DALAM PEMROGRAMAN**



OLEH:

AKHIRATUNNISA HASANAH (2411089008)

PROGRAM STUDI D4 TEKNOLOGI REKAYASA PERANGKAT LUNAK

(ALIH KREDIT)

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI PADANG

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT, yang telah melimpahkan rahmat dan hidayah-Nya. Sehingga makalah yang berjudul "*Refactoring dalam Pemrograman*" ini dapat diselesaikan dengan baik.

Makalah ini disusun untuk memberikan pemahaman mengenai pentingnya refactoring dalam dunia pemrograman, terutama dalam meningkatkan kualitas kode dan efisiensi pengembangan perangkat lunak. Kami menyadari bahwa makalah ini masih jauh dari sempurna, sehingga kritik dan saran yang membangun sangat kami harapkan demi perbaikan di masa mendatang.

Kami mengucapkan terima kasih kepada semua pihak yang telah memberikan dukungan dan bimbingan dalam penyusunan makalah ini. Semoga makalah ini bermanfaat bagi para pembaca, khususnya bagi mereka yang ingin mendalami konsep *refactoring* dalam pemrograman.

Akhir kata, semoga makalah ini dapat memberikan wawasan yang berharga dan menjadi referensi yang bermanfaat bagi pembaca.

Depok, 8 Februari 2025

Akhiratunnisa Hasanah

DAFTAR ISI

HALAMAN JUDUL.....	i
KATA PENGANTAR	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penulisan.....	2
1.4 Manfaat Penulisan.....	3
BAB II PEMBAHASAN	4
2.1 Definisi <i>Refactoring</i>	4
2.2 Kondisi Melakukan <i>Refactoring</i>	4
2.3 Teknik-Teknik <i>Refactoring</i>	5
2.4 Manfaat <i>Refactoring</i>	7
2.5 Tantangan Dalam <i>Refactoring</i>	7
2.6 Implementasi <i>Refactoring</i> dalam 2 bahasa pemrograman (<i>Java</i> dan <i>Javascript</i>).....	8
2.6.1 Implementasi pada Bahasa Java.....	8
2.6.2 Implementasi <i>Refactoring</i> pada Bahasa <i>Javascript</i>	10
BAB III PENUTUP	14
3.1 Kesimpulan	14
DAFTAR PUSTAKA	15

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pengembangan perangkat lunak, kualitas kode menjadi aspek yang sangat penting untuk menjamin keberlanjutan dan kemudahan pemeliharaan sebuah sistem. Seiring berkembangnya perangkat lunak, kode program seringkali mengalami penambahan fitur, perbaikan *bug*, dan modifikasi yang dapat mengakibatkan penurunan kualitas. Kondisi ini dapat menyebabkan kode menjadi sulit dipahami, sulit dimodifikasi, dan berpotensi menimbulkan *bug* baru di masa mendatang.

Dibutuhkan sebuah solusi untuk mengatasi permasalahan ini. Solusi yang tepat adalah *Refactoring*. Pengimplementasian *refactoring* bertujuan untuk meningkatkan kualitas kode program yang sudah berjalan, sekaligus memfasilitasi proses pengembangan dan pemeliharaan software secara berkelanjutan [1]. Dalam penerapannya, *refactoring* menjadi sebuah pendekatan yang sistematis untuk memastikan kode tetap efisien, bersih serta mudah dipahami oleh tim pengembang. Proses ini tentu memberikan manfaat selain mengurangi kompleksitas kode, solusi ini juga memudahkan tim pengembang dalam menemukan *bug* dan segera memperbaikinya sebelum menjadi masalah yang lebih rumit.

Kualitas *software* tidak hanya dinilai dari produk akhirnya saja, tetapi juga dari seluruh tahapan proses pengembangannya [2]. Pada tahap pengembangan *software*, perubahan kode adalah hal yang tidak dapat dihindari yang disebabkan oleh banyak faktor seperti penambahan fitur baru, perbaikan dari fitur yang sudah ada dan disesuaikan dengan kebutuhan dari pengguna yang terus berkembang dari waktu ke waktu. Kualitas *software* yang buruk dapat menyebabkan masalah baru dalam memastikan keberlanjutan sistem. Kode yang tidak terstruktur dan rumit pada teknologi yang lebih lama akan menyulitkan tim pengembang dalam pengembangan dan pemeliharaan sistem. Jika sistem tidak dapat beradaptasi dengan kebutuhan dan teknologi juga menimbulkan dampak negatif lainnya. Beberapa masalah itu seperti biaya pemeliharaan yang semakin meningkat, penurunan efisiensi pengembangan dan risiko lainnya yang lebih besar [3].

Refactoring adalah proses untuk mengubah susunan kode didalam perangkat lunak tanpa mengubah cara kerjanya. Tujuan dari proses *refactoring* adalah untuk membuat kode lebih rapi,

mudah dibaca dan dikelola setelah adanya proses penambahan fitur, perbaikan *bug* dan perubahan lainnya [4]. *Refactoring* dapat membersihkan kode dari “*code smell*”. *Code smell* adalah indikator yang menandakan adanya kelemahan pada desain dan penulisan kodenya. Menurunnya kemampuan dalam memelihara kode adalah dampak yang ditimbulkan oleh *code smell*. Maka penting untuk mengatasi *code smell* [4]. Beberapa contoh umum dari *code smell* adalah *Long Method* (metode yang terlalu panjang), *Duplicated Code* (kode yang berulang di beberapa bagian), dan *Large Class* (kelas yang memiliki terlalu banyak tanggung jawab).

Dengan mempertimbangkan berbagai manfaat dan tantangan yang terkait, penelitian ini bertujuan untuk mengeksplorasi lebih jauh konsep *refactoring*, teknik-teknik yang umum digunakan, serta implementasinya melalui studi kasus pada dua bahasa pemrograman yang berbeda. Harapannya, penelitian ini dapat memberikan wawasan yang lebih mendalam tentang cara menerapkan *refactoring* dalam pengembangan perangkat lunak.

1.2 Rumusan Masalah

Berdasarkan penjabaran latar belakang diatas dapat disusun lima rumusan masalah sebagai berikut:

1. Bagaimana *Refactoring* dapat meningkatkan kualitas kode dalam pengembangan perangkat lunak?
2. Apa saja teknik *Refactoring* yang sering digunakan dalam pengembangan perangkat lunak?
3. Apa saja manfaat *Refactoring* dalam pemeliharaan dan pengembangan perangkat lunak?
4. Apa saja tantangan *Refactoring* dalam meningkatkan efisiensi pengembangan dan pemeliharaan perangkat lunak?
5. Bagaimana implementasi dari *refactoring* pada bahasa pemrograman?

1.3 Tujuan Penulisan

Tujuan dari penulisan makalah ini adalah untuk memberikan informasi dan pengetahuan tentang *Refactoring* dan untuk memenuhi salah satu tugas Ujian Akhir Semester Mata Kuliah Konstruksi dan Evolusi Perangkat Lunak.

1.4 Manfaat Penulisan

Berikut ini adalah beberapa manfaat dari penulisan makalah *Refactoring* ini:

1. Meningkatkan pemahaman mengenai konsep *Refactoring*.
2. Menjelaskan teknik-teknik yang ada dalam *Refactoring*.
3. Untuk mengetahui apa saja manfaat dari *Refactoring*.
4. Untuk mengetahui apa saja tantangan dari *Refactoring*.
5. Untuk mengetahui seperti apa penerapan *refactoring* pada Bahasa pemrograman yang berbeda.

BAB II PEMBAHASAN

2.1 Definisi *Refactoring*

Refactoring adalah proses untuk mengubah susunan kode didalam perangkat lunak tanpa mengubah cara kerjanya. Tujuan dari proses *refactoring* adalah untuk membuat kode lebih rapi, mudah dibaca dan dikelola setelah adanya proses penambahan fitur, perbaikan *bug* dan perubahan lainnya [4]. Jadi yang diubah hanyalah kodenya saja, hasil akhirnya tetap sama.

Selain bertujuan untuk meningkatkan kualitas kode, *refactoring* juga dikenal sebagai proses restrukturisasi perangkat lunak yang bertujuan untuk mengurangi kesalahan (*error*), seperti menghilangkan duplikasi kode atau "*bad code smell*." *Refactoring* juga berkaitan dengan upaya memperbaiki struktur kode dan dokumentasi agar lebih mudah dipahami, sehingga memudahkan pengembang dalam melakukan pengembangan lebih lanjut selama proses evolusi perangkat lunak [4].

Meningkatkan efisiensi dari struktur kode merupakan tujuan utama dari *refactoring*, sehingga *refactoring* bukanlah tentang menambahkan fitur baru ataupun memperbaiki *bug* pada sistem. *Refactoring* ini sering dipakai untuk pengembangan perangkat lunak untuk memastikan bahwa kode tetap bersih dan mudah untuk di atur selama proses pengembangan. Dengan demikian, proses ini akan mempermudah melakukan penyesuaian terhadap perubahan dimasa depan.

2.2 Kondisi Melakukan *Refactoring*

Refactoring adalah teknik penting dalam pengembangan perangkat lunak untuk meningkatkan kualitas kode. Berikut adalah beberapa kondisi yang mengharuskan kita untuk melakukan *refactoring*:

1. Kode program sulit dipahami [3] memerlukan *refactoring* agar lebih mudah dimengerti dan dikelola.
2. kompleksitas tinggi [3]. Kode dengan tingkat kerumitan yang tinggi, ditandai dengan banyak ketergantungan dan logika yang rumit, perlu di-*refactor* agar lebih mudah dipelihara.

3. Sebelum menambahkan fitur baru [3]. Jika penambahan fitur baru menjadi sulit dan memakan waktu lama akibat kode yang kompleks, *refactoring* diperlukan untuk mempermudah pengembangan di masa depan.
4. Saat mengatasi *bug* dan *code smell*. *Refactoring* saat mengatasi *bug* ini untuk memproteksi munculnya masalah yang sama dimasa mendatang.

2.3 Teknik-Teknik *Refactoring*

Didalam penerapannya, terdapat beberapa teknik yang digunakan dalam *refactoring*. Teknik-teknik ini dapat digunakan untuk menyederhanakan struktur kode sehingga mudah dibaca, mengurangi kompleksitas dan redundansi. Teknik-teknik ini akan membantu pengembang dalam mengatur urang kode agar lebih efisien, mudah didalam pemeliharannya, dan mudah dalam penyesuaiannya dengan perubahan dimasa depan. berikut adalah beberapa yang dipakai didalam *refactoring*:

1. *Extract Method*

Dalam praktik *refactoring*, *extract method* merupakan teknik untuk mengorganisir ulang kode dengan memindahkan sekelompok instruksi ke dalam metode baru yang terpisah. Pendekatan ini membantu menciptakan kode yang lebih terstruktur dengan memisahkan fungsi-fungsi spesifik, yang pada akhirnya meningkatkan *maintainability* dan *readability* program [5].

2. *Extract Class*

Extract Class adalah cara merapikan kode program dengan memindahkan beberapa fungsi dari satu kelas ke kelas yang baru. Tujuannya adalah agar kode lebih mudah dibaca dan bisa dipakai lagi di tempat lain. Teknik ini biasanya dilakukan jika ada sekelompok fungsi yang saling berkaitan dan punya tugas khusus yang berbeda dari kelas utamanya [6].

3. *Rename Variable*

Rename Variable adalah teknik penamaan ulang variabel menjadi lebih jelas dan mudah dipahami sesuai dengan fungsinya. Dengan menggunakan teknik ini, kode program akan lebih mudah dibaca dan dipahami oleh pengembang.

4. *Remove Dead Code*

Remove Dead Code adalah teknik *refactoring* yang dilakukan dengan cara menghapus kode yang tidak digunakan didalam sistem. Hal ini sangat penting untuk dilakukan demi menjaga kebersihan kode pada sistem. Dampak dari *dead code* yang tidak dihapus adalah menumpuknya kode-kode yang tidak dibutuhkan pada sistem sehingga bisa membuat tim pengembang kebingungan dan menambah kompleksitas sisten.

5. *Simplify Conditionals*

Simplify Conditionals adalah teknik *refactoring* yang dilakukan dengan cara menyederhanakan logika kondisional yang rumit [7]. Penyederhanaan ini salah satunya dapat dilakukan dengan cara mengganti kondisi *if-else* dengan *ternary operator*.

6. *Move Method*

Move Method atau *Move Field* adalah teknik yang dipakai ketika sebuah metode atau *field* lebih banyak dipakai di kelas lain dibandingkan di kelas tempatnya berada. Dengan memindahkan metode atau *field* tersebut ke kelas yang lebih sering menggunakannya, ketergantungan antar kelas bisa dikurangi, sehingga struktur kode menjadi lebih rapi dan efisien [7].

7. *Inline Method*

Inline Method adalah teknik *refactoring* yang digunakan saat sebuah metode terlalu sederhana, sehingga isinya lebih jelas daripada nama metodenya sendiri [7]. Dalam kasus seperti ini, kita bisa menghapus metode tersebut dan langsung menggunakan kode di dalamnya pada tempat di mana metode itu dipanggil.

8. *Replace Conditional with Polymorphism*

Ketika sebuah program memiliki kondisi yang menentukan tindakan berdasarkan jenis objek, maka setiap bagian dari kondisi tersebut bisa dipindahkan ke metode di dalam *subclass*. Dengan cara ini, metode utama diubah menjadi abstrak, sehingga setiap *subclass* bisa menentukan perilaku masing-masing tanpa perlu menggunakan banyak pernyataan kondisi [8][9]. teknik ini bertujuan untuk mengurangi kompleksitas kode dengan

menggantikan struktur **if-else** atau **switch-case** yang berulang dengan mekanisme *polimorfisme*. Dengan menggunakan *inheritance* dan *overriding*, kode menjadi lebih mudah dipelihara, lebih fleksibel untuk perubahan, serta mengurangi ketergantungan antar komponen dalam sistem.

2.4 Manfaat *Refactoring*

Beberapa manfaat dari *refactoring* adalah sebagai berikut:

1. Meningkatkan keterbacaan kode, *refactoring* akan membuat kode menjadi bersih, mudah dibawa dan dipahami.
2. Mempermudah pemeliharaan dan pengembangan kode. Kode yang terstruktur mempermudah pengembangan dan penambahan fitur baru. Pengembang dapat dengan mudah mengintegrasikan fitur baru tanpa mengganggu fungsionalitas yang sudah ada.
3. Meningkatkan performa sistem. Dalam beberapa kasus, *refactoring* dapat meningkatkan performa aplikasi dengan mengoptimalkan algoritma atau mengurangi penggunaan sumber daya yang tidak perlu.
4. Mengurangi risiko kesalahan dan *bug*. *Refactoring* membantu mengurangi *technical debt* dengan memperbaiki struktur kode dan membuatnya lebih sesuai dengan praktik terbaik. Kode yang sederhana dan mudah dipahami cenderung memiliki lebih sedikit *bug*.
5. Mengurangi duplikasi kode, dengan penerapan beberapa teknik *refactoring* pada kode ini dapat mengurangi redundansi.
6. Meningkatkan fleksibilitas kode. membuat kode lebih fleksibel terhadap perubahan tanpa memerlukan modifikasi besar.

2.5 Tantangan Dalam *Refactoring*

Dalam penerapannya, *refactoring* memiliki beberapa tantangan. Dibawah ini adalah beberapa tantangan yang sering muncul saat pengembang melakukan *refactoring*:

1. Membutuhkan waktu dan sumber daya. Saat berhadapan dengan kode yang lumayan rumit, maka pengembang akan membutuhkan lebih banyak waktu dan usaha, kadang hasilnya pun tidak selalu sesuai.
2. *Refactoring* dapat merusak fungsionalitas dari kode yang sudah ada. Hal ini dapat terjadi jika pengembang kurang hati-hati dalam melakukan *refactoring*.

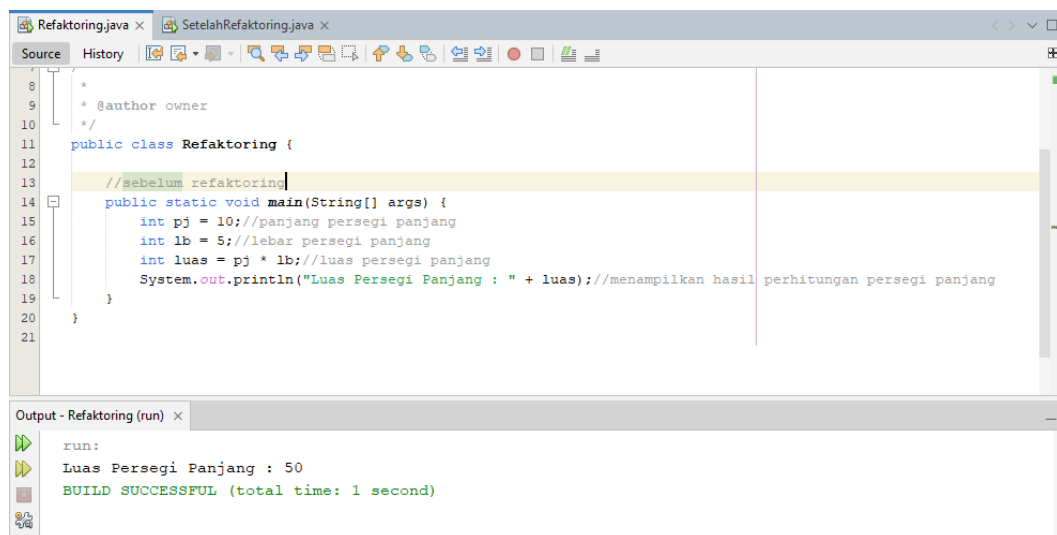
3. Kompleksitas dari internal pernakat lunak.
4. Menjaga agar kode tetap mudah dalam pemeliharaannya.

2.6 Implementasi *Refactoring* dalam 2 bahasa pemrograman (*Java* dan *Javascript*)

Refactoring adalah proses mengubah struktur kode tanpa mengubah fungsionalitas eksternalnya. Tujuan utamanya untuk meningkatkan kualitas kode sehingga kode menjadi mudah untuk dibaca, dipelihara dan dikembangkan. Dalam studi kasus ini, kita akan melihat bagaimana *refactoring* dapat diterapkan pada kode sederhana pada bahasa *Java* dan *Javascript*.

2.6.1 Implementasi pada Bahasa Java

- a. Kode java sebelum *refactoring* pada java:



```

8
9
10
11
12
13
14
15
16
17
18
19
20
21

```

```

*
* @author owner
*/
public class Refactoring {

    //sebelum refactoring
    public static void main(String[] args) {
        int pj = 10; //panjang persegi panjang
        int lb = 5; //lebar persegi panjang
        int luas = pj * lb; //luas persegi panjang
        System.out.println("Luas Persegi Panjang : " + luas); //menampilkan hasil perhitungan persegi panjang
    }
}

```

```

run:
Luas Persegi Panjang : 50
BUILD SUCCESSFUL (total time: 1 second)

```

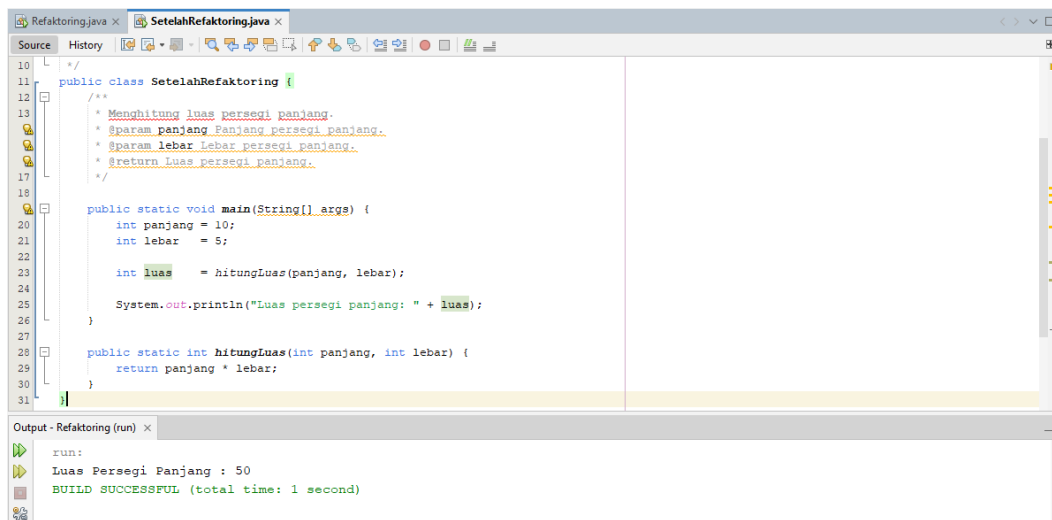
Gambar 1. Kode sebelum di *refactoring* pada Java

Seperti yang tampak pada gambar 1 diatas bahwa kode berjalan dengan baik. Kode diatas sudah dilengkapi dengan komentar yang memberikan informasi kepada tim pengembang mengenai apa fungsi dari masing-masing variabel yang ada. Akan tetapi, kode ini memiliki kekurangan yaitu kode perhitungannya yang tidak terstruktur. Hal ini karena perhitungan luas dilakukan langsung didalam metode *main*, sehingga sulit untuk digunakan kembali ditempat lain. Untuk mengatasi kekurangan ini maka harus dilakukan *refactoring* pada kode tersebut.

b. Proses *refactoring*

Untuk mengatasi kekurangan pada kode sebelumnya, maka *refactoring* harus dilakukan dengan langkah-langkah berikut ini:

1. Mengganti nama variabel “pj” dan “lb” menjadi “panjang” dan “lebar” agar lebih mudah dipahami. Maka pada tahap ini kita menerapkan teknik *refactoring* yaitu *rename variable*.
2. Memisahkan kode perhitungan luas persegi panjang ke dalam metode terpisah. Dibagian ini akan dibuat metode baru yang bernama **hitungLuas** yang akan bertanggung jawab untuk melakukan perhitungan luas persegi panjang. Pada tahap ini menggunakan teknik *Extract Method*.
3. Kode setelah di *refactoring*:



Gambar 2. Kode setelah di *refactoring* pada Bahasa Java

Setelah dilakukan *refactoring*, fungsi **hitungLuas** dapat digunakan ditempat lain. Dengan langkah ini, kode menjadi lebih rapi, mudah dibaca dan dipelihara. variabel “panjang” yang sebelumnya bernama “pj” dan juga variabel “lebar” yang awalnya bernama “lb”, sekarang lebih mudah dipahami sesuai dengan fungsi dari variabel tersebut.

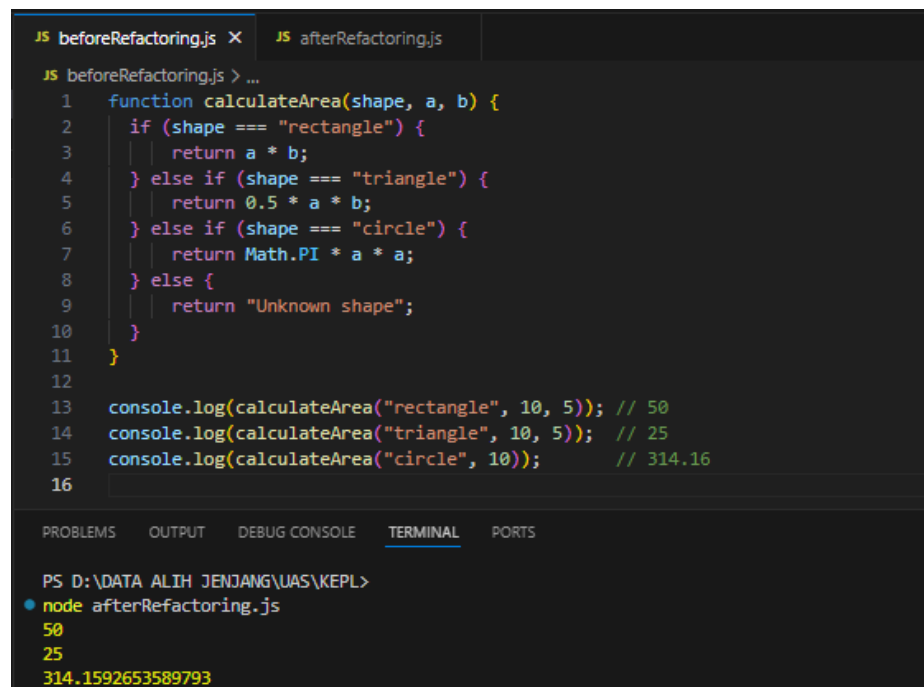
Dari proses diatas dapat ditarik beberapa perbandingan dari kode sebelum dan sesudah *refactoring* pada bahasa Java:

Tabel 1. Perbandingan antara sebelum dan sudah *refactoring* pada Java

No	Faktor Perbandingan	Sebelum <i>refactoring</i>	Sesudah <i>refactoring</i>
1	Penamaan variabel	pj, lb	panjang, lebar
2	Komentar	Ada	Ada
3	Struktur Kode	Tidak terstruktur	Terstruktur
4	Kemudahan penggunaan kembali	Sulit	Mudah
5	Teknik yang digunakan	-	<i>Rename variable, Extract method</i>

2.6.2 Implementasi *Refactoring* pada Bahasa Javascript

a. Kode sebelum di *refactoring*



```

JS beforeRefactoring.js X JS afterRefactoring.js
JS beforeRefactoring.js > ...
1 function calculateArea(shape, a, b) {
2   if (shape === "rectangle") {
3     return a * b;
4   } else if (shape === "triangle") {
5     return 0.5 * a * b;
6   } else if (shape === "circle") {
7     return Math.PI * a * a;
8   } else {
9     return "Unknown shape";
10  }
11 }
12
13 console.log(calculateArea("rectangle", 10, 5)); // 50
14 console.log(calculateArea("triangle", 10, 5)); // 25
15 console.log(calculateArea("circle", 10)); // 314.16
16
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\DATA ALIH JENJANG\UAS\KEPL>
● node afterRefactoring.js
50
25
314.1592653589793

```

Gambar 3. Kode sebelum di *refactoring* pada Bahasa Javascript

Kode pada gambar 3 digunakan untuk menghitung luas berbagai bentuk seperti persegi panjang, segitiga dan lingkaran, kodenya pun sudah berjalan dengan baik. Namun, kode ini memiliki beberapa kekurangan yang dapat diperbaiki dengan *refactoring*, berikut beberapa poin yang harus di *refactoring*:

1. Kode diatas menggunakan beberapa kondisi **if-else** yang menyebabkan kode harus memeriksa setiap bentuk satu-persatu sehingga hal ini juga akan menyulitkan didalam pengelolaannya jika sewaktu waktu pengembang ingin menambahkan bentuk baru.
2. Perhitungan luas dilakukan langsung didalam fungsi utama, sehingga ini akan menyulitkan untuk digunakan kembali ditempat lain.
3. Beberapa parameter tidak fleksibel untuk digunakan, contohnya pada lingkaran yang hanya membutuhkan satu parameter jari-jari, namun di kode tetap menerima dua parameter yang bisa menimbulkan kebingungan.
4. Penggunaan string untuk mencocokkan bentuk akan membuat kode lebih berpotensi untuk memunculkan sebuah kesalahan, contohnya jika ada salah ektik dalam nama bentuk.

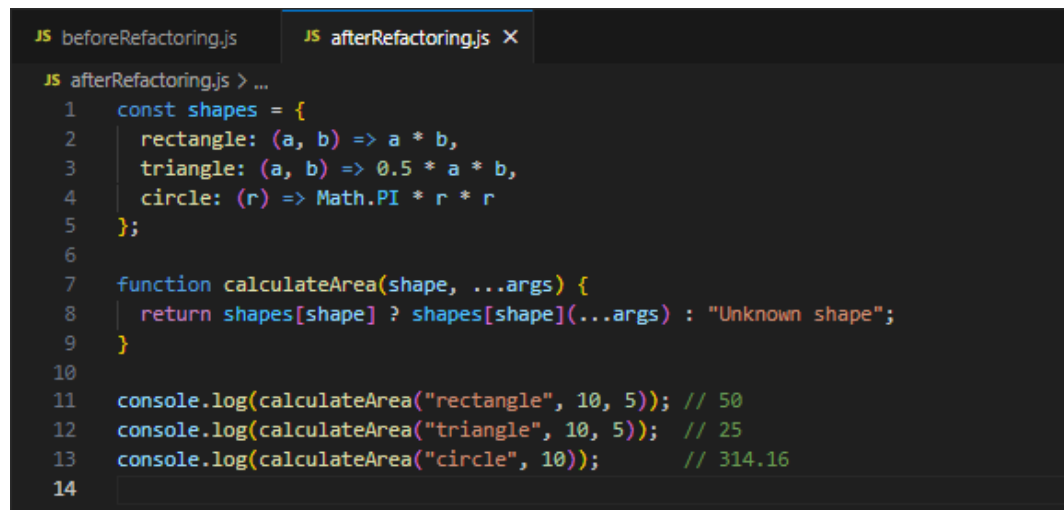
Untuk mengatasi hal-hal diatas, maka perlu dilakukan *refactoring* pada kode tersebut.

b. Proses *refactoring*

Untuk mengatasi kekurangan pada kode sebelumnya, maka *refactoring* harus dilakukan. Berikut ini adalah proses *refactoring* yang dilakukan pada kode program Bahasa pemrograman *Javascript* diatas:

1. Perhitungan luas untuk masing-masing bentuk dipisahkan ke dalam fungsi-fungsi yang lebih kecil yang disimpan dalam objek **shapes**. Maka pada tahap ini kita menerapkan teknik *refactoring* yaitu *extract method*. Hal ini akan mengurangi terjadinya kode berulang dan membuat kode lebih bersih sehingga mudah untuk dibaca dan dipahami.
2. Sebelumnya, kode menggunakan banyak **if-else** untuk memeriksa bentuk yang dipilih. Setelah *refactoring*, kode menggantinya dengan *lookup table* menggunakan objek *shapes*, sehingga setiap bentuk cukup dipanggil langsung dari objek berdasarkan kunci (`shapes[shape]`). Pada bagian ini menggunakan teknik *Replace Conditional with Polymorphism (Replace Conditional with Lookup Table)*.

3. Dengan memisahkan perhitungan ke dalam objek `shapes`, kode menjadi lebih mudah dipahami. Jika ingin menambahkan bentuk baru, cukup tambahkan properti baru dalam objek `shapes` tanpa mengubah struktur utama fungsi `calculateArea`.
4. Menggunakan `...args` dalam parameter `calculateArea` agar fleksibel dalam menangani jumlah parameter yang berbeda. Ini akan meningkatkan fleksibilitas fungsi dan mengurangi potensi *bug* yang disebabkan oleh jumlah parameter yang tidak sesuai.



```

JS beforeRefactoring.js JS afterRefactoring.js X
JS afterRefactoring.js > ...
1  const shapes = {
2    rectangle: (a, b) => a * b,
3    triangle: (a, b) => 0.5 * a * b,
4    circle: (r) => Math.PI * r * r
5  };
6
7  function calculateArea(shape, ...args) {
8    return shapes[shape] ? shapes[shape](...args) : "Unknown shape";
9  }
10
11 console.log(calculateArea("rectangle", 10, 5)); // 50
12 console.log(calculateArea("triangle", 10, 5)); // 25
13 console.log(calculateArea("circle", 10)); // 314.16
14

```

Gambar 4. Kode javascript yang sudah di *refactoring*

Gambar 4 menunjukkan kode yang sudah di *refactoring* pada bahasa pemrograman *Javascript*. Kode menjadi lebih rapi dan mudah dibaca dan dipahami. Kodenya pun menjadi lebih terstruktur.

Dari proses diatas dapat ditarik beberapa perbandingan dari kode sebelum dan sesudah *refactoring* pada Bahasa Javascript yang terdapat pada tabel 2 dibawah ini:

Tabel 2. Perbandingan antara sebelum dan sesudah *refactoring* pada *Javascript*.

No	Faktor Perbandingan	Sebelum <i>refactoring</i>	Sesudah <i>refactoring</i>
1	Struktur Kode	Menggunakan banyak kondisi if-else untuk setiap bentuk	Menggunakan <i>Look-up table</i> (objek shapes) untuk memanggil fungsi perhitungan berdasarkan nama bentuk.
2	Kemudahan Penambahan Bentuk Baru	Sulit	Mudah
3	Fleksibilitas Parameter	Tidak fleksibel. Menerima dua parameter (a, b) untuk semua bentuk, padahal beberapa bentuk (misalnya lingkaran) hanya membutuhkan satu parameter.	Fleksibel. Menggunakan <i>rest parameter</i> (...args) untuk menerima jumlah parameter yang berbeda sesuai kebutuhan masing-masing bentuk.
4	Potensi Kesalahan	Tinggi. Penggunaan <i>string</i> untuk nama bentuk meningkatkan risiko salah ketik.	Rendah. Penggunaan <i>lookup table</i> meminimalkan risiko salah ketik karena nama bentuk didefinisikan sebagai kunci dalam objek.
5	Teknik <i>Refactoring</i> yang Diterapkan	-	<i>Extract Method</i> (memisahkan perhitungan luas ke dalam fungsi terpisah) dan <i>Replace Conditional with Polymorphism</i> (mengganti kondisi if-else dengan <i>lookup table</i>).

Melalui implementasi *refactoring* pada contoh program Java dan JavaScript, kita dapat melihat bagaimana teknik ini membantu meningkatkan kualitas kode tanpa mengubah fungsionalitasnya. Dengan menerapkan prinsip-prinsip *refactoring* seperti simplifikasi struktur kode, penghapusan duplikasi, dan peningkatan keterbacaan, kode menjadi lebih mudah dipahami, dipelihara, serta dikembangkan dimasa mendatang.

BAB III PENUTUP

3.1 Kesimpulan

Refactoring adalah proses untuk mengubah susunan kode didalam perangkat lunak tanpa mengubah cara kerjanya. Tujuan dari proses *refactoring* adalah untuk membuat kode lebih rapi, mudah dibaca dan dikelola setelah adanya proses penambahan fitur, perbaikan *bug* dan perubahan lainnya. Dari implementasi *refactoring* pada dua bahasa pemrograman, yaitu Java dan JavaScript, dapat disimpulkan beberapa manfaat utama:

1. Meningkatkan keterbacaan kode – Dengan menggunakan teknik seperti *Rename Variable* dan *Extract Method*, kode menjadi lebih mudah dipahami oleh pengembang lain.
2. Mempermudah pemeliharaan dan pengembangan kode – Struktur kode yang lebih terorganisir memungkinkan pengembang untuk lebih mudah menambahkan fitur baru tanpa merusak sistem yang ada.
3. Meningkatkan performa sistem. - Dalam beberapa kasus, *refactoring* dapat mengoptimalkan algoritma atau mengurangi penggunaan sumber daya yang tidak perlu, sehingga meningkatkan efisiensi sistem.
4. Mengurangi risiko kesalahan dan *bug* – Dengan struktur yang lebih bersih dan penggunaan metode yang sesuai, potensi *bug* dapat diminimalisir, serta mengurangi *technical debt* yang bisa menyebabkan masalah di masa depan.
5. Mengurangi duplikasi kode – Dengan menerapkan *Extract Method* dan *Replace Conditional with Polymorphism*, kode menjadi lebih modular dan mengurangi redundansi.
6. Meningkatkan fleksibilitas kode – Penggunaan teknik seperti *Lookup Table* pada JavaScript membuat kode lebih fleksibel terhadap perubahan tanpa memerlukan modifikasi besar.

Dalam implementasinya pada Java dan JavaScript, *refactoring* terbukti membantu dalam membuat kode lebih mudah dipahami, dan lebih efisien. Penerapan teknik *refactoring* seperti *Extract Method* dalam Java dan penggunaan *Lookup Table* dalam JavaScript menunjukkan bahwa kedua bahasa ini memiliki pendekatan yang berbeda tetapi tetap memberikan hasil yang serupa dalam meningkatkan kualitas kode.

DAFTAR PUSTAKA

- [1] A. B. Baqais and M. Alshayeb, “Automatic software refactoring: a systematic literature review,” *Softw. Qual. J.*, vol. 28, no. 2, pp. 459–502, 2020, doi: 10.1007/s11219-019-09477-y.
- [2] R. K. Hapsari and M. J. Husen, “Estimasi Kualitas Perangkat Lunak Berdasarkan,” *Semin. Nas. Sains dan Teknol. Terap. III*, pp. 425–434, 2015.
- [3] R. Andriani, P. N. Sabrina, dan H. Ashaury, "Metode Refactoring Untuk Meningkatkan Kualitas Maintainability Pada Sistem Informasi Puskesmas," *Jurnal TEKNO KOMPAK*, vol. 19, no. 1, hlm. 227–239, 2023.
- [4] R. Nindyasari, “METODE NON- HEURISTIC UNTUK DETEKSI REFACTORING NON-SOURCE CODE (SYSTEMATIC LITERATURE REVIEW),” vol. 6, no. 2, pp. 361–366, 2015.
- [5] N. Tsantalis and A. Chatzigeorgiou, “Identification of extract method refactoring opportunities for the decomposition of methods,” *J. Syst. Softw.*, vol. 84, no. 10, pp. 1757–1782, 2011, doi: 10.1016/j.jss.2011.05.016.
- [6] M. Fokaefs, N. Tsantalis, E. Stroulia, and A. Chatzigeorgiou, “Identification and application of Extract Class refactorings in object-oriented systems,” *J. Syst. Softw.*, vol. 85, no. 10, pp. 2241–2260, 2012, doi: 10.1016/j.jss.2012.04.013.
- [7] Almogahed, A., & Omar,M. (2021). Refactoring techniques for improving software quality: A practitioners’ perspectives. *Journal of Information and Communication Technology*, 20(4), 511-539. <https://doi.org/10.32890/jict2021.20.4.3>
- [8] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts, *Refactoring: Improving the Design of Existing Code*, First ed.: Addison-Wesley Longman Publishing Co., Inc., 1999.
- [9] M. Fowler. (2008, November 18, 2008). *Refactoring Catalog: Refactorings in Alphabetical Order*. Available: <http://www.refactoring.com/catalog/index.html>